Stored procedure və function fərqi

Bu dərs əsasən Microsoft SQL Server və onun T-SQL dilinə aiddir. SQL cədvəllərdəki məlumatı sorğulamaq və dəyişmək üçün dildir. T-SQL SQL Server-in dəyişənlər, prosedurlar və idarəetmə əmrləri əlavə edən SQL variantıdır. Eyni “function” sözünün PostgreSQL-də bəzi məhdudiyyətləri fərqlidir; bir sistemin qaydasını o birinə köçürmək olmaz.

İlkin materiallar `MSSQL_Functions_AZ.pdf`, `MSSQL_Functions_AdventureWorks.pdf`, `MSSQL_Tehsil_Materiallari.pdf` və onun ikinci variantıdır. İlk iki sənəddə funksiya və procedure müqayisəsi artıq var. Bu dərs həmin bazanı genişləndirir, qaydaların sərhədini, nümunələrdəki sütun uyğunsuzluğunu, paralel yazma riskini və plan iddialarını dəqiqləşdirir.

Mündəricat

Stored procedure və function fərqi **1**

1 Rutin parametr və nəticə 2

2 Eyni meyarlarla müqayisə 2

3 Scalar function nümunəsi 2

4 Cədvəl qaytaran iki function növü 3

5 Procedure üç fərqli nəticə verə bilər 4

6 Transaction və xəta sərhədi 4

7 Performans və təhlükəsizlik mifləri 5

8 Seçim məşqləri və cavablar 6

9 İlkin SQL nümunələrində konkret düzəlişlər 6

10 Rəsmi mənbələr 6

1 Rutin parametr və nəticə

Routine, rutin, bazada adla saxlanan və yenidən çağırılan kod vahididir. **Stored procedure** əməliyyat və ya əmrlər ardıcıllığı icra edən saxlanmış rutindir. **Function** SQL ifadəsinin bir hissəsi kimi istifadə edilə bilən dəyər və ya cədvəl qaytaran rutindir. **UDF**, User Defined Function, istifadəçinin yaratdığı funksiyadır; `SUM` kimi hazır funksiyadan fərqlənir.

Parametr çağırış zamanı verilən giriş və ya çıxış dəyişənidir. `@CustomerId` T-SQL parametr adıdır. **Scalar** bir dəyər, məsələn `42` və ya `15.50` deməkdir. **Result set** sorğudan qayıdan sətir və sütunlar toplusudur. **Schema** baza obyektlərini qruplaşdıran addır; `dbo.Orders` içində `dbo` schema, `Orders` cədvəldir. Schema-nı göstərmək obyektin dəqiq tanınmasına kömək edir.

Procedure “sifarişi təsdiqlə, ehtiyatı azalt, audit qeydi yaz” kimi əməliyyatlar üçün uyğundur. Function “bu məbləğin vergisi nədir?” və ya “bu müştərinin sifariş cədvəlini ifadə kimi ver” sualı üçün uyğundur. Audit baş verən əməliyyatın sonradan yoxlama üçün qeydə alınmasıdır.

2 Eyni meyarlarla müqayisə

Meyar	SQL Server davranışı
Çağırış	Procedure `EXEC dbo.Name ...`; scalar function `SELECT dbo.Name(...)`; table-valued function `FROM dbo.Name(...)`.
Nəticə	Procedure result set, OUTPUT parametr və tam ədəd return status verə bilər. Function müqaviləsindəki scalar və ya cədvəli qaytarır.
Sorğuya qoşulma	Function SELECT və ya FROM daxilində uyğun formada işlənir. Procedure normal halda JOIN mənbəyi kimi birbaşa yazılmır.
Məlumat dəyişmə	Procedure cədvəllərdə INSERT UPDATE DELETE edə bilər. T-SQL UDF baza vəziyyətini dəyişən yan təsirlər üçün istifadə edilmir.
Transaction və xəta	Procedure uyğun transaction və TRY/CATCH məntiqi daşıya bilər. T-SQL UDF daxilində bu imkanlar məhduddur.
Dinamik SQL və müvəqqəti cədvəl	Procedure istifadə edə bilər. T-SQL UDF daxilində dynamic SQL və temp table olmaz; table variable mümkündür.

Side effect, yan təsir, nəticə qaytarmaqdan əlavə xarici vəziyyəti dəyişməkdir. **Dynamic SQL** SQL mətninin işləmə vaxtında qurulub icra edilməsidir. **Temp table** `#Name` ilə müvəqqəti cədvəldir. **Table variable** `@t TABLE (...)` formasında cədvəl tipli lokal dəyişəndir. Bu iki vasitənin optimizasiya və davranış fərqləri var, eyni şey sayılmır.

3 Scalar function nümunəsi

T-SQL

```
CREATE OR ALTER FUNCTION dbo.NetAmount
(
    @Gross decimal(12,2),
    @Discount decimal(12,2)
)
RETURNS decimal(12,2)
AS
BEGIN
    RETURN @Gross - @Discount;
END;
GO

SELECT dbo.NetAmount(100.00, 15.00) AS Net;
```

Nəticə `85.00` olur. `decimal(12,2)` ümumilikdə 12 rəqəm, vergüldən sonra 2 rəqəm saxlayan dəqiq ədədi tipdir. `RETURNS` nəticə tipini elan edir, `RETURN` funksiyanın dəyərini verir. `AS Net` nəticə sütununa ad verir. `GO` serverə göndərilən T-SQL əmri deyil; SSMS kimi alətlərdə ayrı batch sərhədidir. Batch birlikdə göndərilən əmrlər qrupudur. Tətbiq kodundan göndərəndə GO hissələrini ayrı əmrlərə bölmək lazımdır.

Bu sadə hesablamanın function-a çıxarılması həmişə vacib deyil. Hər çağırışda `Gross - Discount` da kifayət edə bilər. `NULL` naməlum və ya olmayan dəyərdirsə çıxma nəticəsi də NULL ola bilər. `COALESCE` ilk NULL olmayan dəyəri seçir, amma naməlum məbləği avtomatik sıfır saymaq biznes qaydasına uyğun olmalıdır.

Deterministic eyni giriş və eyni nəzərə alınan vəziyyətdə eyni nəticə verən davranışdır. **Nondeterministic** davranışda tarix vaxt və digər dəyişən amillər nəticəni dəyişə bilər. Bütün nondeterministic funksiyalar bütün kontekstlərdə eyni qayda ilə qadağan edilmir; konkret UDF məhdudiyyətini yoxlamaq lazımdır.

4 Cədvəl qaytaran iki function növü

TVF, Table Valued Function, cədvəl nəticəsi verir. **Inline TVF** bir SELECT ifadəsi ilə müəyyən edilir. Optimizer, yəni icra üsulunu seçən baza komponenti, onun məntiqini əsas sorğu ilə birlikdə təhlil edə bilir.

T-SQL

```
CREATE OR ALTER FUNCTION dbo.CustomerOrders
(
    @CustomerId int
)
RETURNS TABLE
AS
RETURN
(
    SELECT Id, CustomerId, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId
);
GO

SELECT Id, Total
FROM dbo.CustomerOrders(7)
WHERE Total > 100;
```

Bu nümunə `dbo.Orders(Id, CustomerId, Total)` cədvəli mövcud olduğunu nəzərdə tutur. Funksiyanın nəticəsi xaricdə əlavə filtr və JOIN üçün istifadə olunur. **Multi-statement TVF** isə cədvəl dəyişəni elan edir, bir neçə əmrlə onu doldurur və qaytarır. Daha çox məntiqə imkan verir, lakin sətir sayının qiymətləndirilməsi

və optimizasiya məhdudiyyətləri yarada bilər. Yeni versiyalardakı optimizasiya imkanları səbəbindən “həmişə yavaşdır” demək də doğru deyil.

5 Procedure üç fərqli nəticə verə bilər

T-SQL

```
CREATE OR ALTER PROCEDURE dbo.GetCustomerOrders
    @CustomerId int,
    @OrderCount int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Total
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    SELECT @OrderCount = COUNT(*)
    FROM dbo.Orders
    WHERE CustomerId = @CustomerId;

    RETURN 0;
END;
GO

DECLARE @Count int, @Status int;
EXEC @Status = dbo.GetCustomerOrders
    @CustomerId = 7,
    @OrderCount = @Count OUTPUT;

SELECT @Count AS OrderCount, @Status AS Status;
```

Birinci SELECT sifarişlərdən result set verir. `@Count` OUTPUT parametr ilə sayı alır. `@Status` isə procedure-un return status-unu alır; bu nümunədə 0 uğuru bildirmək üçün seçilib. “Procedure heç nə qaytarmır” buna görə yanlış cümlədir. Eyni zamanda procedure `RETURN` ilə mətn və ya cədvəl qaytarmır, tam ədəd status verir.

`DECLARE` dəyişən elan edir. `COUNT(*)` sətirləri sayır. `SET NOCOUNT ON` təsirlənən sətir sayı barədə ayrıca protokol mesajlarını azaldır; SELECT nəticəsini gizlətmir. Onu hər ssenaridə böyük sürət artımının səbəbi kimi göstərmək olmaz. İki SELECT arasında başqa transaction dəyişiklik edə bilər; nəticə ilə sayın eyni anı əks etdirməsi tələb olunursa uyğun transaction və isolation seçimi lazımdır.

6 Transaction və xəta sərhədi

Transaction bir neçə dəyişiklik üçün bütövlük sərhədidir. **Atomicity** onun əməliyyatlarının birlikdə təsdiqlənməsi və ya geri alınmasıdır. `COMMIT` təsdiqləyir, `ROLLBACK` geri alır. `TRY/CATCH` icra xətasını tutmaq üçün bloklardır. `THROW` xətanı çağırana ötürür. `XACT_STATE()` transaction-un aktiv və təsdiqlənə bilən olub olmadığını bildirir.

T-SQL

```
-- Bu fraqment oz transaction-unun sahibidir.
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE dbo.Orders
    SET Status = 'Confirmed'
    WHERE Id = @OrderId;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

`XACT_ABORT ON` bir çox runtime xətasında transaction-un tam ləğv olunmasına səbəb olur; bütün xəta növləri üçün universal qayda deyil. Bu fraqmentdə `@OrderId` xaricdən verilmiş parametrdir və `Status` sütunu mövcuddur. UPDATE sıfır sətrə təsir edərsə bu avtomatik xəta deyil; lazım gəlsə `@@ROWCOUNT` yoxlanmalıdır. `@@ROWCOUNT` son əmrin təsir etdiyi sətir sayıdır.

Procedure başqa transaction içindən də çağırıla bilər. Daxili `COMMIT` xarici transaction-u müstəqil təsdiqləmir; iç-içə transaction sayğacı və sahiblik qaydaları nəzərə alınmalıdır. Bu sadə nümunəni ümumi nested transaction şablonu kimi köçürmək olmaz. Bir transaction xaricə göndərilmiş email-i geri ala bilmir.

7 Performans və təhlükəsizlik mifləri

Execution plan sorğunun hansı operatorlarla icra olunacağını göstərən plandır. **Plan cache** hazırlanmış planların təkrar istifadə üçün yaddaşda saxlandığı yerdir. Procedure çox vaxt ilk icrada uyğun plan hazırlayır, sonradan onu təkrar istifadə edə bilər. Schema və statistika dəyişiklikləri və başqa səbəblər recompilation, yəni yenidən plan qurulması yarada bilər. Parametrli adi SQL də plan təkrar istifadəsindən yararlana bilər. “Procedure əvvəlcədən tam kompilyasiya olunub həmişə daha sürətlidir” dəqiq deyil.

Parameter sniffing plan hazırlanarkən konkret parametr dəyəri əsasında qiymətləndirmə aparılmasıdır. Bir müştərinin 2, başqasının 2 milyon sifarişi varsa eyni plan hamı üçün optimal olmaya bilər. Bu normal optimizasiya mexanizminin bəzən uyğun olmayan nəticəsidir; həlli ölçmə və versiyanın imkanları ilə seçilir.

Scalar UDF böyük nəticədə hər sətir üçün əlavə iş yarada bilər. SQL Server-in uyğun versiya və şərtlərində scalar UDF inlining həmin məntiqi əsas plana daxil edə bilər. Inlining ayrıca function çağırışını ifadəyə açmaqdır. Dəstək və uyğunluq olmadığı halda eyni qazancı gözləmək olmaz.

SQL injection istifadəçi mətninin məlumat əvəzinə icra edilən SQL hissəsinə çevrilməsidir. Procedure daxilində istifadəçi mətnini SQL-ə birləşdirmək yenə injection yarada bilər. Dəyərləri parametr kimi ötürmək lazımdır. `sp_executesql` dinamik mətnlə ayrıca parametr dəyərlərini qəbul edən sistem procedure-udur. Cədvəl və sütun adı kimi identifikatorlar adi dəyər parametri deyil; onların məhdud icazəli siyahısı və düzgün ad qaçırılması lazımdır.

İcazəni procedure səviyyəsində vermək müəyyən təhlükəsizlik dizaynlarında faydalıdır, amma “tam qorunma” zəmanəti deyil. İstifadəçinin həqiqətən hansı məlumatı oxuya bildiyi, dynamic SQL və icra konteksti ayrıca nəzərə alınmalıdır.

8 Seçim məşqləri və cavablar

Sifarişi və ehtiyatı birlikdə dəyişmək? Procedure və ya tətbiqin idarə etdiyi transaction məntiqli başlanğıcdır. Function yan təsirli iş vahidi üçün uyğun deyil.

Cədvələ JOIN edib təkrar istifadə olunan filtr vermək? Inline TVF uyğun ola bilər. Procedure birbaşa FROM mənbəyi kimi normal istifadə olunmur.

Hər sətirdə 18 faiz hesablamaq? Sadə SQL ifadəsi və ya uyğun scalar function mümkündür. Çox sətirdə planı ölç. Təkrar istifadə ehtiyacı ilə çağırış xərcini birlikdə qiymətləndir.

PostgreSQL-də də function məlumat dəyişə bilməzmi? Bu, SQL Server T-SQL UDF qaydasıdır. PostgreSQL funksiyaları müəyyən şərtlərdə yazma əməliyyatları apara bilər. Procedure və function transaction nəzarəti qaydaları da həmin sistemdə ayrıca öyrənilməlidir.

Məşq. 3 sifarişlik Orders cədvəlində TVF və procedure yaz. TVF-i WHERE ilə daralt. Procedure-un result set-ini, OUTPUT sayını və return status-unu ayrı oxu. Nəticə olaraq bu üç qaytarılma kanalını bir-birindən izah etməyi bacarmalısan.

9 İlkin SQL nümunələrində konkret düzəlişlər

`MSSQL_Functions_AZ.pdf` 7-ci səhifədə `AktivİşçiLər` inline TVF-i İşçiID, TamAd, Email, Maaş və ŞöbəAdı qaytarır. Ardınca gələn JOIN isə `AI.VəzifəID` istifadə edir. Bu sütun function nəticəsində olmadığı üçün həmin sorğu işləməz. İşçilər cədvəlində həqiqətən belə xarici açar varsa onu function-un SELECT siyahısına əlavə etmək lazımdır. Yoxdursa JOIN doğru əlaqə açarı ilə yenidən qurulmalıdır. TVF xarici sorğuya yalnız qaytardığı sütunları təqdim edir, daxildəki cədvəlin bütün sütunlarını avtomatik açmır.

Həmin sənədin 8-ci səhifəsində multi-statement hesabatda `OrtaMaaş` sütunu elan edilir, amma göstərilən INSERT və UPDATE-lər onu doldurmur. Bu halda sütun NULL qalır; hesabatda orta məbləğ görünəcəyi nəticəsi çıxmır. Mənalı orta hesabı üçün mənbə dəyəri və qruplaşdırma qaydası əlavə edilməli və ya işlənməyən sütun nəticədən çıxarılmalıdır. `AVG` orta qiymət hesablayan SQL funksiyasıdır; `GROUP BY` sətirləri seçilmiş açara görə qruplaşdırır.

Müştəri əlavə etmə nümunələrində əvvəl `IF EXISTS`, sonra INSERT ilə email təkrarı yoxlanır. İki paralel sorğu eyni anda “yoxdur” nəticəsini alıb ikisi də insert edə bilər. Qayda email-in unikal olmasıdırsa uyğun UNIQUE constraint və ya unique index bazada bunu qorumalıdır. Tətbiq duplicate xətasını istifadəçiyə uyğun cavaba çevirə bilər. Duplicate eyni açarın təkrarıdır. Adi əvvəlcədən yoxlama rahat mesaj verir, amma paralel yazmada constraint-i əvəz etmir. NULL və email müqayisə qaydaları da həmin biznes tələbinə görə müəyyən edilir.

10 Rəsmi mənbələr

- Microsoft Learn - CREATE FUNCTION
- Microsoft Learn - Create user defined functions
- Microsoft Learn - CREATE PROCEDURE

Yoxlama tarixi 28 oktyabr 2026. SQL nümunələri izah üçündür, real bazada icra edilməyib. Cədvəllər və parametrlər göstərilən şərtlərlə ayrıca hazırlanmalıdır.